Unit-2 Arrays, Strings, List, Tuple and Dictionary

Array

An array is an object that stores a group of elements with same datatype. The main advantage of an array is to store and process a group of elements easily.
An array can store only one type of data. It means, we can store only integer type elements or only float type elements into an array. But we cannot store one integer, one float and one character type element into the same array.
An array can increase or decrease their size dynamically. It means we need not declare the size of an array. When element are added, it will increase its size and when the element are removed, it will automatically decrease its size in memory.

Advantages of arrays:

- Arrays are similar to list. The main difference is that arrays can store only one type of elements; list can store different type of element.
- When dealing with huge number of element array use less memory then list.
- Arrays offer faster execution then list.

Creating an array:

Syntax:

```
Arrayname=array(type code, [elements])
```

Typecode	C Type	Minimum size in byte
'i'	Signed integer	2
'I'	Unsigned integer	2
'f'	Floating point	4
'd'	Double precision floating point	8
'u'	Unicode character	2

Importing the array module:

There are three ways to import the array module into our program.
1st way:

```
import array
```

When we import the array module, we are able to get the 'array' class of that module that helps us to create an array.

```
import array
a= array.array('i',[1,2,3,4])
print(a)
```

Here, the first 'array' represents a module name and the next represents the class name for which object is created.

2nd way:

```
import array as ar
b= ar.array('i',[1,2,3,4,5])
print(b)
```

Here 'ar' is an alias of array module.

3rd way:

```
from array import *
c= array('i',[1,2,3,4,5,6])
print(c)
```

The meaning of this statement is import all (classes, objects, variables etc) from the array module into our program. So there is no need to mention the module name before our array name while creating.

Example: Program in which characters are use

```
from array import *
c= array('u',['a','c','d','e','b'])
for ch in c:
        print(ch)
```

Example: Creating an array copying type and data from other array

```
from array import *
c= array('u',['a','c','d','e','b'])
d=array(c.typecode,(a for a in c))
for ch in d:
    print(ch)
```

<u>Indexing and Slicing on Arrays</u>

<u>Indexing:</u>

An index represents the position number of elements in the array. For example, when we create the array like this: a = array('i',[10,20,30,40,50]). Python interpreter allocates 5 blocks of memory.

10	20	30	40	50
X[0]	X[1]	X[2]	X[3]	X[4]

Here, 0,1,2, etc, are representing the position numbers of the elements. So in general we can use 'i' to represent the position of any element. This 'i' is called 'index' of the array.

Example: Retrieving array element using array index and for loop

```
from array import *
a=array('i',[10,20,30,40,50])
n=len(a) #finds number of elments in an array
print(n)
for i in range(n):
  print(a[i],end=" ")
```

Example: # Retrieving array element using array index and while loop

```
from array import *
a=array('i',[10,20,30,40,50])
n=len(a) #finds number of elments in an array
print(n)
i=0
while i<n:
  print(a[i],end=' ')
  i+=1
```

Slicing:

A slice represents a piece of the array. Slicing is useful to retrieve a range of elements. The general format of a slice is:

```
Array_name[start: stop:stride]
```

We can eliminate any one or any two from above three.

Example: Slicing

```
from array import *
c= array('u',['a','t','m','i','y','a','u','n','i','v','e','r','s','i','t','y'])
a=c[1:16]
a=c[0:]
a=c[:16]
a=c[:4]
a=c[2:5]
a=c[-16:]
a=c[-16:-5]
a=c[0:16:2]
print(a)
```

Example: Slicing using for loop

```
from array import *
c= array('u',['a','t','m','i','y','a','u','n','i','v','e','r','s','i','t','y'])
for i in c[0:16]:
   print(i,end='')
```

Processing the Arrays

The arrays class of arrays module in Python offers methods to process the arrays easily. The programmers can easily perform certain operations by using these methods.

Array Methods:

Method	**Description**
a.append(x)	Adds an element x at the end of the array a.
a.count(x)	Returns the numbers of occurrences of x in the array a.
a.extend(x)	Appends x at the end of the array a.
a.fromfile(f,n)	Reads n items from the file object f and appends to the end of the array a.
a.fromlist(lst)	Appends items from the lst to the end of the array.
a.fromstring(s)	Appends items from string s to the end of the array a.
a.index(x)	Returns the position number of the first occurrence of x in the array.
a.insert(i,x)	Inserts x in the position i in the array.
a.pop(x)	Removes the item x from the array a and returns it.
a.pop()	Remove last item from the array a.
a.remove(x)	Removes the first occurrence of x in the array a.
a.reverse()	Reverses the order of elements in the array a.
a.tofile(f)	Writes all elements to the file f.
a.tolist()	Converts the array ‘a’ into the list.
a.tostring()	Converts the array into a string.

Using the above methods:

```
from array import*
#Creating an array named ar with type integer
ar=array('i',[1,2,3,4,5])
print('The initial value of an array is:',ar)

#Appending values to the array
ar.append(6)
ar.append(7)
print('An array value after appending:',ar)

#Inserting the value at the desired position
ar.insert(0,0)
ar.insert(4,3)
print('An array value after inserting:',ar)

#Removing an element from the array
ar.remove(5)
print('After removing the value:',ar)

#Removing the last element from the array
a=ar.pop()
print('The poped element was:',a)
print('Elements after the pop are: ',ar)
#Finding position of element
b=ar.index(3)
print('The first occurence of element 3 is at: ',b)
#Convert an array into a list
lst=ar.tolist()
print('Values of list: ', lst)
print('Values of array: ',ar)
```

Example: Taking marks of students, finding total of marks and percentage

```
from array import*
str=input('Enter marks: ').split(' ')
marks=[int(num) for num in str]
total=0
for a in marks:
    print(a)
    total=total+a
print('The total score is: ', total)
l=len(marks)
percent=total/l
print('The Percentage is: ', percent)
```

Example: Searching an element from the array:

```
from array import*
a=array('i',[])
print('How many elements you want to enter?:',end='')
n=int(input())
for i in range(n):
        print('Enter element: ',end='')
        a.append(int(input()))
print('Elements of array are: ',a)

print('Which element you want to search?: ',end='')
srch=int(input())
try:
        posi=a.index(srch)
        print('found at position ', posi)
except ValueError:
        print('Element you are searching is not found in the array')
```

Types of Arrays:

Single dimensional arrays: These arrays represent only one row or one column of elements. For example, marks obtained by a student in 5 subjects can be written as:

```
Marks=array('i', [20,30,40,50,60])
```

The above array contains only one row (marks of one student) hence; it is called single dimensional array or one dimensional array.

Multi-dimensional arrays: These arrays represent more than one row and more than one column of elements. For example, marks obtained by 3 students in 5 subjects can be written as:

```
Marks=array('i', [[20,30,40,50,60],
                  [20,30,40,50,60],
                  [20,30,40,50,60]])
```

Python supports only single dimensional array. But, we can create multi-dimensional arrays using third party packages like 'numpy' (numerical python).

Working with Array using numpy

Numpy is a package that contains several classes, functions, variables etc. to deal with scientific calculations in Python. We can create single dimensional as well as multi-dimensional arrays using numpy. Numpy contains a large library of mathematical functions. The arrays which are created using numpy are called n dimensional array.

If numpy module not found than install it using 'pip install numpy' in cmd.

To work with numpy, we must first import numpy module into our Python programs as:

```
Import numpy
```

Example: Create a simple array using numpy

```
#import numpy
#ar=numpy.array([10,2,3,4,5])
#import numpy as np
#ar=np.array([10,2,3,4,5])
# from numpy import*
# ar=array([10,2,3,4,5])
```

```
# a1=array([1,2.5,3,4,5])
# a2=array(['a','b','c'])
# a3=array(['apple','banana','chocolate'])
# print(ar)

# print(a1)
# print(a2)
# print(a3)

#Creating array using linspace
#syntax: linspace(start,stop,n)
        #start means: starting element,
        #stop means: ending element,
        #n means: no. of parts elements should be divided (by default 50)
# from numpy import*
# a=linspace(2,10,5)
# print(a)

#Creating array using logspace
#syntax: logspace(start,stop,n)
        #start means: starting element (10^start),
        #stop means: ending element (10^stop),
        #n means: no. of parts elements should be divided (by default 50)
# from numpy import*
# a=logspace(2,10,5)
# print(a)

#Creating array using arange() function
#syntax: arange(start,stop,stepsize)
from numpy import*
a=arange(1,10,2)
print(a)
```

<u>Operations on Array:</u>

```
#Operations on Array
#(same can be done in normal array)
        from numpy import*
        ar=array([40,30,20,-10])
        print(ar)
        print('Adding 5 to each element of array: ',ar+5)
        print('Subtracting 5 from each element of array: ',ar-5)
        print('Multiplying 5 to each element of array: ',ar*5)
        print('Dividing each element of array by 5: ',ar/5)
        print('The biggest element of an array: ',max(ar))
        print('The smallest element of an array: ',min(ar))
        print('The sum of all element of an array: ',sum(ar))
        print('The average of an array: ',mean(ar))
```

```
#Comparing Arrays
        from numpy import*
        a=array([2,3,4,5])
        b=array([5,3,4,2])
        print('Hello',a==b)
        print('Hello',a>b)
        print('Hello',a>=b)
        print('Hello',a<b)
        print('Hello',a<=b)
        print('Hello',a!=b)
```

```
#Compare corresponding elements of two arrays and retrieve the biggest elements.
        from numpy import*
        a=array([1,2,3,4,5,6,7])
        b=array([2,1,5,3,7,4,6])
        c=where(a>b,a,b) #where is the function
        print(c)
```

The where() function can be used to create a new array based on whether a given condition is True or False. The syntax of the where() is: array=where(condition, expression1, expression2).
The use of where() is demonstrated in the above example.

```
#A program to retrieve non zero elements from an array
        from numpy import*
        a=array([2,3,0,4,5,6,0,7])
        b=nonzero(a) #nonzero is the function
        print(a)
        print(a[b])
```

The nonzero() function is used to know the positions of elements which are non zero.

```
#A program to create a view of an existing array.
        from numpy import*
        a=array([10,20,30,40,50,60,70])
        b=a.view() #Use of view() function
        print('Elements of an array a are: ',a)
        print('Elements of new array are: ',b)
        b[2]=300
        print('Elements of an array after modifying value in b array are: ',a)
        print('Elements of new array after modifying value are: ',b)
```

A view() function create another array same as an existing array. The original array and the newly created arrays will share different memory locations. If the newly created array is modified, the original array will also be modified and vice versa. View() function creates a copy which is known as ‘Shallow copying’. The use of view() function is demonstrated in the above program.
In case we want both arrays but, modifying one array must not have an effect on other one than we may use copy() function. The use of copy() is demonstrated in the below program.

```
#A program to create a view of an existing array.
        from numpy import*
        a=array([10,20,30,40,50,60,70])
        b=a.copy() #Use of copy() function
        print('Elements of an array a are: ',a)
        print('Elements of new array are: ',b)
        b[2]=300
        print('Elements of an array after modifying value in b array are: ',a)
        print('Elements of new array after modifying value are: ',b)
```

Slicing and Indexing numpyArrays:

Slicing

Slicing refers to extracting a range of elements from the array. The format of slicing operation is given here: *arrayname[start:stop:stepsize]*

Here, the default value for 'start' is 0, for 'stop' is n(number of elements) and 'stepsize' is 1. Counting starts from 0th position.

Example:

```
        from numpy import*
        a=array([10,11,12,13,14,15,16,17])
        print(a)
        #Retrieving 1st element to 6th element alternatively
        print(a[1:6:2])
        #Retrieving all the elements
        print(a[:]) # or print(a[::])
        #Retrieve all elements from 3rd element
        print(a[3:])
        #Retrieve all elements from 3rd element and with alternatively
        print(a[3::2])
```

Indexing

Indexing refers to the locations of the elements. By specifying the location numbers from 0 onwards till n-1, we can refer to all elements as a[i] where i can change from 0 to n-1.

Example:

```
        from numpy import*
        a=array([10,11,12,13,14,15,16,17])
        print(a)
        i=0
        while(i<len(a-1)):
                print(a[i],end=' ')
                i=i+1
```

Attributes of an Array

The attribute:

The 'ndim' attribute represents the number of dimensions of the array. For a single dimensional array, it will display 1 and for two dimensional array it will display 2.

```
from numpy import*
a=array([10,11,12,13,14,15,16,17])
print(a.ndim)
b=array([[10,11,12,13],[14,15,16,17]])
print(b.ndim)
```

The shape attribute:

The 'shape' attribute give the shape of an array. The shape is a tuple listing the number of elements along each dimensions.

```
from numpy import*
a=array([10,11,12,13])
print(a.shape)
b=array([[10,11,12,13],[14,15,16,17]])
print(b.shape)
print(b)
b.shape=(4,2)
print(b)
print(b.shape)
```

The size attribute

The 'size' attribute gives the total number of elements in the array.

```
from numpy import*
a=array([10,11,12,13])
print(a.size)
b=array([[10,11,12,13],[14,15,16,17]])
print(b.size)
```

The dtype attribute

The 'dtype' attribute gives the datatype of the elements in the array.

```
from numpy import*
a=array([10,11,12,13])
print(a.dtype)
b=array([10.2,11,12,13])
print(b.dtype)
```

The reshape() method

The 'reshape()' method is useful to change the shape of an array.

```
from numpy import*
a=array([10,11,12,13,14,15])
a=a.reshape(2,3)
print(a)
a=a.reshape(3,2)
print(a)
```

The flatten() method

The 'flatten()' method is useful to collapse the elements of an array into one dimension.

```
from numpy import*
a=array([10,11,12,13,14,15])
a=a.reshape(2,3)
print(a)
a=a.flatten()
print(a)
```

Multi-dimensional arrays

The 2D arrays, 3D arrays etc. are called multi-dimensional arrays.

```
from numpy import*
a=array([[10,11,12,13],[14,15,16,17]])
print(a)
```

The internal memory allocated to all the elements would be in a single row containing 8 blocks (as there are 8 elements in the array). The elements are stored in the contiguous memory location as shown below:

10	11	12	13	14	15	16	17
[0][0]	[0][1]	[0][2]	[0][3]	[1][0]	[1][1]	[1][2]	[1][3]

The reshape() function

This function is useful to convert a 1D array into a multidimensional array. It would be written as below: reshape(arrayname,(n,r,c)).

Here 'n' represents number of array in the resultant array, 'r' represents row and 'c' represents number of columns.

Example:

```
from numpy import*
a=array([1,2,3,4,5,6,7,8,9,10,11,12])
print(a)
b=reshape(a,(4,3))
print(b)
c=reshape(a,(2,3,2))
print(c)
```

Slicing in Multi-dimensional arrays

The format of slice operator is: arrayname[start:stop:stepsize].

The default value for start is 0, for stop it is 'n'(number of elements) and for stepsize it 1.

Example: Slicing

```
from numpy import*
a=array([[1,2,3],[4,5,6],[7,8,9]])
print(a)
print(a[0,:])#0th row
print(a[2,:])#2nd row
print(a[:,0])#0th column
print(a[0:1,0:1])#element at 0th row 0th column
print(a[2:3,1:2])
```

String

```
s1='Welcome to Atmiya University, Rajkot' #Both s1 and s2 have same effect
s2="Welcome to Atmiya University, Rajkot" #Both s1 and s2 have same effect
s3='Welcome to "Atmiya University", Rajkot' #To display the quote in the o/p
s4='Welcome to \t"Atmiya University", \nRajkot' #Using escape characters
s5=r'Welcome to \t"Atmiya University", \nRajkot' #Creating a 'raw string' by putting r before the string
print(s1)
print(s2)
print(s3)
print(s4)
print(s5)
s6=u'\u9811'
print(s6)
print(len(s1)) #Finding the length of a string
print(s1*2) #Repeating the string
s7='Atmiya'
s8='University'
s=s7+s8
print(s)
```

0	1	2	3	4	5
A	T	M	I	Y	a
-6	-5	-4	-3	-2	-1

```
#A program to access each element of a string in forward and reverse order (using while)
        str='Atmiya University'
        n=len(str)
        i=0
        while i<n:
                print(str[i], end=' ')
                i=i+1
        print()

        #In the reverse order
        i=1
        n=len(str)
        while i<=n:
                print(str[-i],end='')
                i=i+1


#A program to access each element of a string in forward and reverse order (using for loop)
        str='Atmiya University'
        i=0
        for i in str:
                print(i,end=' ')
        print()
```

```
        #In reverse order
        for i in str[::-1]:
                print(i,end=' ')

#Slicing the string
        str= 'Atmiya University'
        print(str[0:17:1])
        print(str[0:17:2])
        print(str[-1::-1])

#Checking whether string exist in the main string
        str=input('Enter the main string: ')
        substr=input('Enter the string you wanted to find: ')
        if substr in str:
                print(substr,'Is available in the main string')
        else:
                print(substr,'Is is not available in the main string')

#To check whether entered string is palindrome
        str1=input('Enter the string: ')
        str2=(str1[-1::-1])
        print(str2)
        if(str1==str2):
                print('The string is Palindrome')
        else:
                print('The string is not a Palindrome')


#Counting sub-string in a string
#Format: stringname.count(substring,beg,end)
#Counting in the range given
        str='New India New Dreams'
        n=str.count('New',0,10)
        print(n)
#Counting From the whole string
        str='New India New Dreams'
        n=str.count('New')
        print(n)

#Replacing a String
        str='Good Morning!'
        s1='Morning'
        s2='Evening'
        print(str)
        str=str.replace(s1,s2)
        print(str)
```

```
#Changing case of a string
        str='Your Future is Bright'
        print(str.upper())
        print(str.lower())
        print(str.swapcase())
        print(str.title())
```

#String Testing Methods

Method	Description
isalnum()	This method returns True if all the members in the string are combination of alphabet and numeric (A to Z, a to z, 0 to 9) otherwise it returns False.
isalpha()	This method returns True if all the members in the string are alphabet (A to Z, a to z) otherwise it returns False.
isdigit()	This method returns True if all the members in the string are numeric (0 to 9) otherwise it returns False.
islower()	This method returns True if all the members in the string are alphabets in lower case (a to z) otherwise it returns False.
isupper()	This method returns True if all the members in the string are alphabets in lower case (A to Z) otherwise it returns False.

Example:

```
        mobile_number=input('Enter Your mobile number: ')
        print(mobile_number.isdigit())
        if mobile_number.isdigit() != True:
                print('Enter valid mobile number')
```

```
#Sort a string into alphabetical order
        str=[]
        n=int(input('How many strings you want to enter?: '))
        for i in range(n):
                print('Enter the string: ', end=' ')
                str.append(input()) #use of append method
        print(str)
        str1=str.sort() #use of sort method
        print(str)
        for i in str:
                print(i)
```

```
#Program to search the string from the group of strings and also find its position
        str=[]
        n=int(input('How many strings you want to enter?: '))
        for i in range(n):
                print ('Enter the string: ', end=' ')
                str.append(input()) #use of append method
        print(str)
        #Ask user for the string to be searched
```

```
        srch=input('Enter string to search: ')
        #Searching
        flag=False
        for i in range(len(str)):
                if srch==str[i]:
                        print('The searched string found at the position: ',i+1)
                        flag=True
        if flag==False:
                        print('The string you are searching is not found')

#Working with characters
        str='Atmiya'
        ch=str[0]
        print(ch)
        ch=str[4]
        print(ch)
        ch=str[0:3]
        print(ch)
#Program to know the type of a character
        str=input('Enter a character: ')
        ch=str[0]
        if ch.isalpha():
                print('It is an alphabet')
                if ch.isupper():
                        print('String entered is in Upper case')
                else:
                        print('String entered is in Lower case')
        else:
                print('It is a number')
```

List

```
#Program to create list using range function
        #format of range:range(start, stop, stepsize)
#0 to 7
        lst1=range(8)
        for i in lst1:
                print(i, ',', end='')
#8 to 15
        print()
        lst2=range(8,16)
        for i in lst2:
                print(i, end=' ')
#Odd numbers between 1-10
        print()
        lst3=range(1,10,2)
        for i in lst3:
                print(i, end=' ')
```

```
#Updating the list
        lst=list(range(1,8))
        print(lst)
#Appending the list (appending means adding value at the last position)
        lst.append(11)
        print(lst)
#Updating the elment of the list
        lst[4]=50
        print(lst)
#Deleting the elment from the list by position
        del lst[4]
        print(lst)
#Deleting the elment from the list by value
        lst.remove(6)
        print(lst)
#Finding index of an element
        a=lst.index(2)
        print(a)
#Clearing all elements from the list
        lst2.clear()
        print(lst2)
#Length of a string
        n=len(lst)
        print(n)
#Display element of list in reverse order
        lst = ['sunday', 'monday', 'tuesday']
        a=lst.reverse() #Use of reverse() method
        print(a)
        print(lst)

#Concatenation of lists
        a=[1,2,3]
        b=['one','two','three']
        print(a+b) #Concatenation by putting '+' between two lists
        #Repetition of list
        a=[1,2,3]
        print(a*3)#Repetition using '*' and a digit after list name

#Membership in Lists
        #Using 'in' and 'not in'
        lst=[1,2,3,4,5,6,7]
        a=7
        b=8
        print(a in lst)
        print(b in lst)
        print(a not in lst)
```

```
        print(b not in lst)
#Aliasing a list
        lst1=[1,2,3,4,5,6,7]
        lst2=lst1
        print(lst1)
        print(lst2)
        lst2[2]=100
        print(lst1)
        print(lst2)
#Cloning a list
        lst1=[1,2,3,4,5,6,7]
        lst2=lst1[:]
        print(lst1)
        print(lst2)
        lst2[2]=100
        print(lst1)
        print(lst2)

#Counting how many times an element is found in the list
        a=[]
        n=int(input('How many elements to insert?: '))
        for i in range(n):
                print ('Enter element: ', end=' ')
                a.append(int(input()))
        print('The list is: ',a)
        find=int(input('Enter an element to count: '))
        cnt=0
        for i in a:
                if (find==i):
                        cnt=cnt+1
        print('{} is found {} times.'.format(find,cnt))

#Finding Common elements in two lists
        flower1=['rose','marigold','mogra']
        flower2=['rose','orchid', 'daisy']
        #Converting it to set
        s1=set(flower1)
        s2=set(flower2)
        s3=s1.intersection(s2)
        print(s3)
        #Converting resultant set to list
        common=list(s3)
        print(common)

#Create a nested list and display its elements
        lst=[1,2,3,[10,20]]
        print('The elements of list: ',lst)
```

```
        print('The first element is: ', lst[0])
        print('The last element is: ', lst[3])
        print('Displaying all elements of the list')
        for i in lst[0:3]:
                print(i)
        for i in lst[3]:
                print(i)

#Create a list which contains square of 1-10
        sqr=[]
        for i in range(1,11):
                sqr.append(i**2)
        print(sqr)
```

Tuple

-Tuple is similar to list, with only one difference that tuple is immutable. We cannot modify its elements.
-Writing elements into the variable without putting any type of brackets will by default create tuple.

```
Tuple
a=(1,) #Check without comma
print(a)
b=1,2,3,'abc','rajkot'
print(b)
#Converting list to tuple
lst=[1,2,3]
print(lst)
tpl=tuple(lst)
print(tpl)
tpl=(1,2,3,4,5,6,7,8)
print(tpl)
print(tpl[1:4])
print(tpl[1:7:2])
tpl=(1,'shrey')
#Storing elements of tuple into seprate variables
        rno,nm=tpl[0:2]
        print(rno)
        print(nm)
#Functions to process tuples (len, min, max, count, index, sorted)
tpl=(1,2,3,4,5,6,7,8)
print(len(tpl))
print(min(tpl))
print(max(tpl))
print(tpl.count(7))
print(tpl.index(7))
print(sorted(tpl))
print(sorted(tpl,reverse=True))#in the reverse order
```

Nested Tuples

```
tpl=(10,20,30,(40,50))
print(tpl)
print(tpl[3])
#Taking details of employees in tuple and sort by salary
        emp=((2,'cbc',30000),(1,'cae',20000),(3,'efg',25000))
        print(emp)
        print(emp[2])
        print(sorted(emp))

#Inserting elements in a tuple
#Since we know that values in tuple cannot be modified
#There is another way to do so
#Below program is also applicable to integers
        names=('a','b','d','e','f')
        print(names)
        #Enter new name to insert and position at which to be enterd
        lst=input('Enter a new name: ')
        new_nm=tuple(lst)
        pos=int(input('Enter the position: '))
        #copy names from 0th position to pos-1
        temp_name=names[0:pos-1]
        print(temp_name)
        #Concatenate new element at pos-1
        temp_name=temp_name+new_nm
        print(temp_name)
        #Concatenate remaining elements of names from pos-1 till end
        names=temp_name+names[pos-1:]
        print(names)

#Modify elements in the existing tuple
        num=(10,20,30,50,50)
        print(num)
        #Accept new element and position number
        lst=[int(input('Enter the new element: '))]
        new_val=tuple(lst)
        pos=int(input('Enter the position: '))
        #Copy from 0th position to pos-1
        temp_val=num[0:pos-1]
        print(temp_val)
        #Concatenate new element at pos-1
        temp_val=temp_val+new_val
        print(temp_val)
        #Concatenating remaining elementsof num from pos-1 till end
        num=temp_val+num[pos:]
        print(num)
```

```
#Delete an element from a particular position
        num=(10,20,30,50,40)
        print(num)
        #Accept position number of the element to delete
        pos=int(input('Enter postion number: '))
        #Copy from 0th to pos-1 into another tuple
        num1=num[0:pos-1]
        num=num1+num[pos:]
        print(num)
```

Dictionary

-A dictionary represents a group of elements arranged in the form key-value pairs. In the dictionary, first element is considered as 'key' and the immediate next value is considered as its 'value'. We cannot using slicing or indexing to retrieve elements

Example:

```
        dict={1:'abc',2:'cde',3:'efg',4:'ghi',5:'ijk'}
        print(dict)

#Operations on Dictionaries
        dict={'name':'abc','id':21,'sal':10000}    #Creating a dictionary
        print('The id of employee is: ', dict['id'])    #retrieving values using keys
        print('The name of employee is: ', dict['name'])    #retrieving values using keys
        print('The salary of employee is: ', dict['sal'])    #retrieving values using keys
        n=len(dict) #To find the length of the dictionary
        print('No. of key value pairs are: ',n)
        dict['sal']=20000   #Updating the salary
        print('The salary of employee is: ', dict['sal'])   #retrieving values using keys
        dict['Dept']='Python development'   #Inseting new key:value
        print(dict)     #Display whole dictionary
        del dict['id']     #Deleting key:value
        print(dict)     #Display whole dictionary
        print('id' in dict)     #To check whether the key is available or not, 'in' and 'not in' is used
        print('id' not in dict)    #To check whether the key is available or not, 'in' and 'not in' is used
```

Dictionary Methods

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

```
#Dictionary Methods
        dict={'name':'abc','id':21,'sal':10000} #Creating a dictionary
        print(dict)
        dict1=dict.copy()   #Copys the content of dict to dict1
        print('value of dict1 is: ',dict1)
        a=dict.keys()      #stores the keys of dict in variable
        print(a)
        a=dict.values()   #stores the values of dict in variable
        print(a)
        print(dict.get('name'))   #Displays the value of specified key
        #print(a)
        dict.update({'sal':20000})
        print(dict)
        # a=dict.items()   #Returns a list containing a tuple for each key value pair
        # print(a)
        dict.pop('id')      #Pops the value of key
        print(dict)
        dict.clear()    #clear(), clears the dictionary and results empty dictionary
        print(dict)

#Program to take subject name and marks from student and give total of all subjects
        marks={}
        print('How many subjects you want to enter? :', end=' ')
        n=int(input())
        for i in range(n):
                print('Enter key: ', end=' ')
                key=input()
                print('Enter value: ',end=' ')
                value=int(input())
                marks.update({key:value})
        print('The value of dictionary is:  ', marks)
#=====================================================
```

```
        tot=sum(cia.values())
        print('Total marks: ',tot)
#=====================================================
        print('Enter name of subject to know its marks: ')
        subject=input()
        marks=cia.get(subject,-1)
        if(marks==-1):
                print('Invalid Subject')
        else:
                print('The marks in {} are {}.'.format(subject,marks))
```

#Converting Lists into dictionary

```
        nm_of_state=['Gujarat','Maharashtra','Rajasthan','MP','Karnataka']
        capital_of_state=['Gandhinagar','Mumbai','Jaipur','Bhopal','Bengaluru']
        #Making dictionary by making 'zip' class object
        z=zip(nm_of_state,capital_of_state)
        d=dict(z)
        print(d)
```

Zip()

The zip() function is useful to convert the lists or any sequences into a zip class object. We can pass 1 or more sequences that can be passed to zip()

#Converting Strings into dictionary

```
        str='1=abc, 2=cde, 3=efg, 4=hij, 5=klm'
        print(str)
        #Break the string when ',' is found and '=' is found
        lst=[]
        for x in str.split(','):
                y=x.split('=')
                lst.append(y)
        #Converting list into dictionary
        #But in the dictionary id and name both would be string
        d=dict(lst)
        print(d)
        #Creating new dictionary to convert id from string to integer
        d1={}
        for key,value in d.items():
                d1[int(key)]=value
        print(d1)
```